Course Module

Digital Electronics / Computer Organization and Architecture

Prepared For

Undergraduate Students

CONTENTS

1. Introduction
2. Latches and Flip-Flops
3. SR Latch
4. Gated SR Latch
5. D Flip-Flop
6. Shift Registers
7. Types of Shift Registers
8. Input / Output (I/O) Organization
9. Interrupts
10. Direct Memory Access (DMA)
11. Buses
12. Pipelining

---

1. INTRODUCTION

Modern digital systems and computer systems depend on two important capabilities:

1. **Storage of binary information**
2. **Transfer and processing of data efficiently**

To achieve these, systems use:

- **memory elements** such as latches and flip-flops,
- **shift registers** for data movement,
- **I/O interfaces** for communication with external devices,
- **interrupts** and **DMA** for efficient data handling,
- **buses** for communication between system components, and
- **pipelining** for improving processor performance.

This lecture note introduces these concepts in a clear and structured manner for undergraduate study and examination.

---

2. LATCHES AND FLIP-FLOPS

2.1 Memory Elements in Digital Systems

Digital systems often require temporary storage of information.
The simplest devices used for storing binary information are:

- **Latches**
- **Flip-Flops**

Both are examples of **sequential circuits**, where the output depends not only on the current input but also on the **previous state**.

2.2 Latch

A **latch** is a **basic memory device** capable of storing **one bit** of binary information.

A latch is generally **level-triggered**, which means its output may change whenever the input changes while the circuit is enabled.

Characteristics of a Latch

- Stores **1 bit**
- Simple sequential circuit
- Output depends on:
    - present input
    - previous output state

2.3 Flip-Flop

A **flip-flop** is a **bistable sequential circuit** that stores **one bit** of information and changes its state only when a **clock pulse** is applied.

A flip-flop is generally **edge-triggered**, making it more suitable for synchronous digital systems.

Characteristics of a Flip-Flop

- Stores **1 bit**
- Controlled by clock pulse
- Used in:
    - registers

- counters
- memory systems

2.4 Difference Between Latch and Flip-Flop

Latch	Flip-Flop
Level-triggered	Edge-triggered
Output changes while enabled	Output changes only at clock edge
Simpler circuit	More controlled and stable
Used in simple memory applications	Used in synchronous systems

---

3. SR LATCH

3.1 Introduction

The **SR latch** is the simplest and most fundamental memory circuit in digital electronics.
It stores **one bit** of data and forms the basis for many other sequential circuits.

3.2 Meaning of SR

- **S = Set**
- **R = Reset**

The latch has:

- **2 inputs** → S and R
- **2 outputs** → Q and $\overline{Q}$

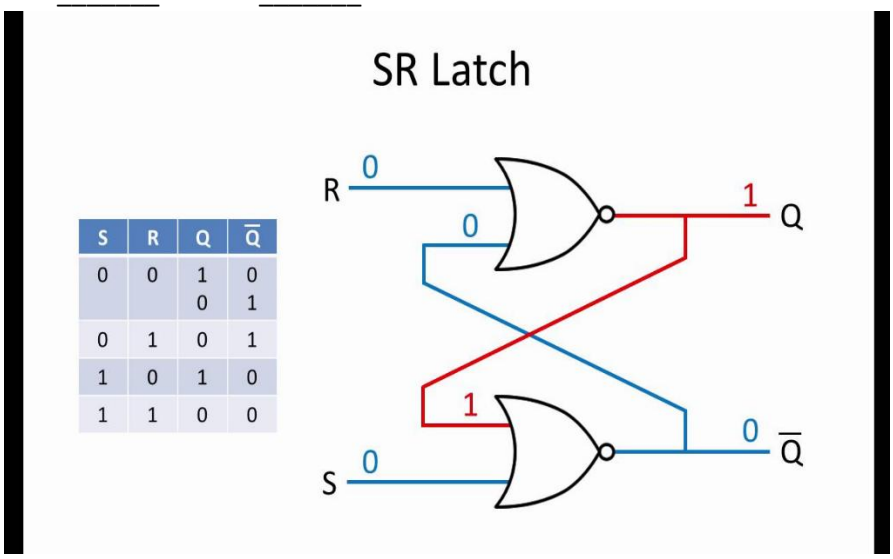
Where:

- **Q** = main output
- $\overline{\mathbf{Q}}$ = complement output

3.3 SR Latch Using NOR Gates

An SR latch can be constructed using **two cross-coupled NOR gates**.

Circuit Diagram



This feedback arrangement allows the latch to **retain its previous state**, thereby storing one bit of information.

3.4 Truth Table of SR Latch

S	R	Q(next)	Operation
0	0	No change	Hold
1	0	1	Set
0	1	0	Reset
1	1	Invalid	Not allowed

3.5 Working of SR Latch

Case 1: S = 1, R = 0 (Set Condition)

The latch is **set**, and the output becomes:

- **Q = 1**
- **$\bar{Q}$ = 0**

This means the latch stores a binary **1**.

Case 2: S = 0, R = 1 (Reset Condition)

The latch is **reset**, and the output becomes:

- **Q = 0**
- **$\bar{Q}$ = 1**

This means the latch stores a binary **0**.

Case 3: S = 0, R = 0 (Hold Condition)

The latch retains the **previously stored value**.

This is the **memory condition**.

Case 4: S = 1, R = 1 (Invalid Condition)

This is an **invalid or forbidden state**.

In a NOR-based SR latch:

- both outputs become **0**
- Q and $\bar{Q}$ are no longer complements

Hence, this condition must be avoided.

3.6 Applications of SR Latch

- Simple memory storage
- Control circuits
- Building block for flip-flops

---

4. GATED SR LATCH

4.1 Need for Gated SR Latch

In a basic SR latch, the output changes immediately whenever the inputs change.
In many digital systems, it is necessary to allow changes **only when required**.

This is achieved by adding an **Enable (E)** input.

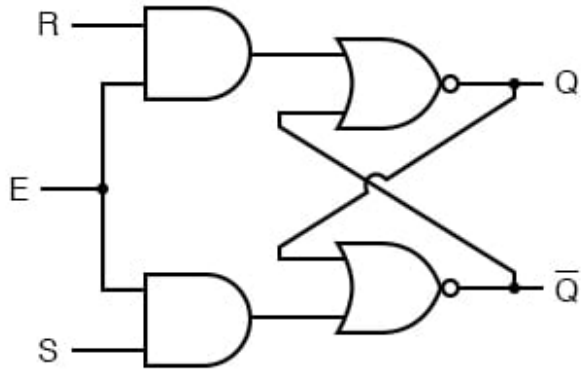
4.2 Definition

A **Gated SR latch** is an SR latch whose operation is controlled by an **Enable input**.

The latch responds to S and R inputs **only when the Enable input is active**.

4.3 Basic Circuit Idea

The inputs **S** and **R** are passed through **AND gates** along with the **Enable signal** before being applied to the SR latch.



E	S	R	Q	$\overline{Q}$
0	0	0	latch	latch
0	0	1	latch	latch
0	1	0	latch	latch
0	1	1	latch	latch
1	0	0	latch	latch
1	0	1	0	1
1	1	0	1	0
1	1	1	0	0

4.4 Truth Table of Gated SR Latch

E S R Output

0 X X No change

1 0 0 Hold

1 1 0 Set

1 0 1 Reset

1 1 1 Invalid

4.5 Working of Gated SR Latch

When E = 0

- the latch is **disabled**
- output remains unchanged
- previous value is retained

When E = 1

- the latch is **enabled**
- it behaves like a normal SR latch

Thus, the **Enable input controls when data can be stored**.

4.6 Importance

The Gated SR latch is useful in circuits where data must be stored only at specific times.

5. D FLIP-FLOP

5.1 Introduction

The SR latch suffers from an invalid condition when both S and R are 1.
To avoid this problem, the **D flip-flop** is used.

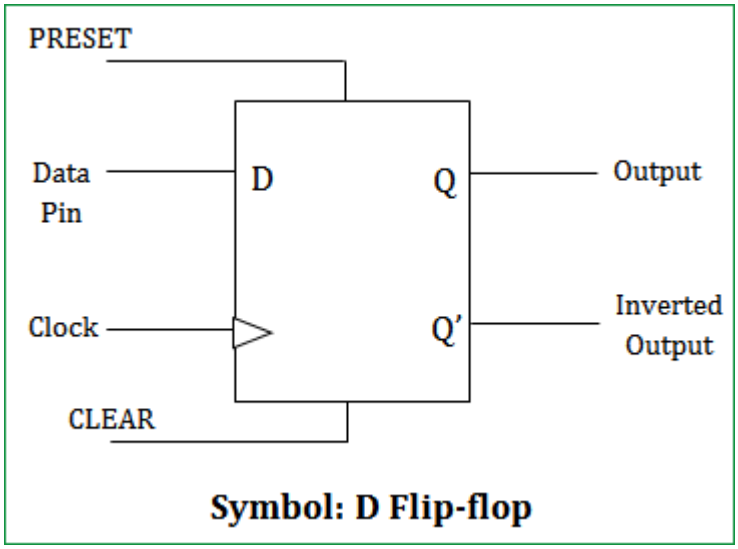
5.2 Definition

A **D Flip-Flop** is a clock-controlled memory element that stores the value present at the **D input** when a **clock pulse** is applied.

Meaning of D

- **D = Data**
  or
- **D = Delay**

5.3 Symbol



Symbol: D Flip-flop

5.4 Truth Table of D Flip-Flop

Clock D Q(next)

↑ 0 0

↑ 1 1

5.5 Working of D Flip-Flop

The output changes only at the **active clock edge**.

- If **D = 1** at the clock pulse → **Q becomes 1**
- If **D = 0** at the clock pulse → **Q becomes 0**

Thus, the D flip-flop stores the input value at the time of the clock pulse.

5.6 Why D Flip-Flop Avoids Invalid State

Internally:

- **S = D**
- **R = $\overline{D}$**

Therefore:

- if D = 1 → S = 1 and R = 0
- if D = 0 → S = 0 and R = 1

Hence:

- **S and R can never both be 1**
- invalid state is eliminated

5.7 Applications of D Flip-Flop

- Registers
- Counters
- Data storage systems
- Shift registers

---

6. SHIFT REGISTERS

6.1 Introduction

A **shift register** is a group of flip-flops connected in such a way that binary data can be:

- stored
- shifted left
- shifted right
- converted between serial and parallel form

Shift registers are very important in digital systems and communication circuits.

6.2 Definition

A **shift register** is a sequential circuit consisting of multiple flip-flops used to **store and shift binary data** under clock control.

6.3 Shifting

Shifting means moving binary data from one flip-flop to the next with each clock pulse.

For example:

- data can move from **FF1 → FF2 → FF3 → FF4**

6.4 Number of Flip-Flops Required

To store **n bits**, we require **n flip-flops**.

Example

To store **4 bits**, we need:

4 flip-flops

---

7. TYPES OF SHIFT REGISTERS

The major types of shift registers are:

1. **SISO** – Serial In Serial Out
2. **SIPO** – Serial In Parallel Out
3. **PISO** – Parallel In Serial Out
4. **PIPO** – Parallel In Parallel Out
5. **Bidirectional Shift Register**

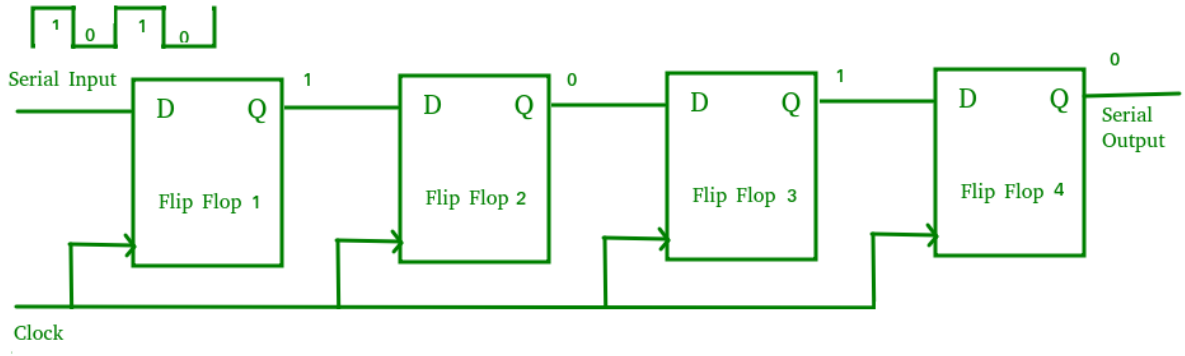
---

8. SERIAL-IN SERIAL-OUT (SISO) SHIFT REGISTER

8.1 Definition

A **Serial-In Serial-Out (SISO)** shift register accepts data **one bit at a time** and produces output **one bit at a time**.

8.2 Block Diagram



8.3 Working

Suppose the input sequence is **1011**.

- At the **first clock pulse**, the first bit enters FF1
- At the **second clock pulse**, the previous bit shifts to FF2
- The next input bit enters FF1
- This continues until the entire bit sequence shifts through the register

Thus, the data is transferred in **serial form**.

8.4 Applications

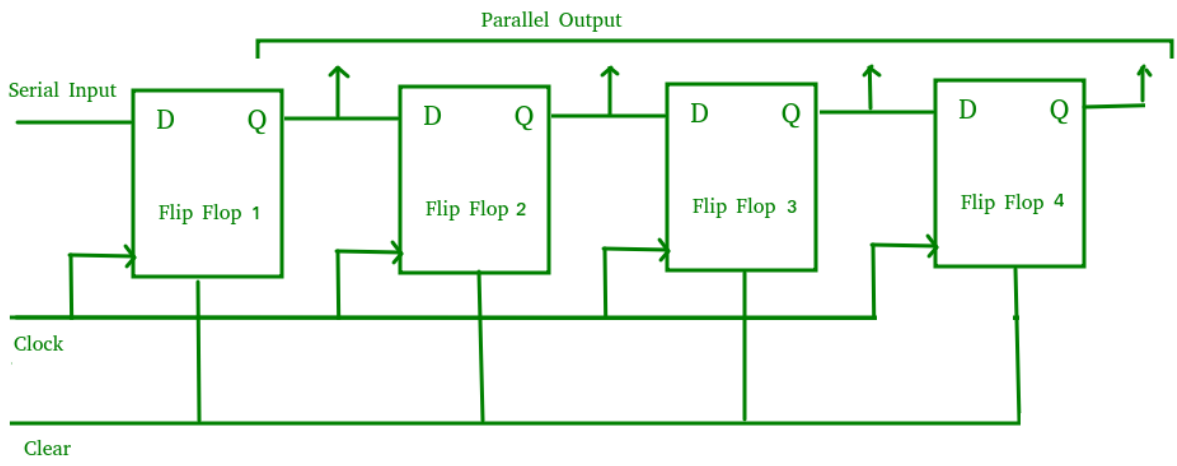
- Serial communication
- Delay circuits
- Sequential data transfer

9. SERIAL-IN PARALLEL-OUT (SIPO) SHIFT REGISTER

9.1 Definition

A **Serial-In Parallel-Out (SIPO)** shift register accepts input data serially and makes it available at the outputs in **parallel form**.

9.2 Block Diagram



9.3 Working

Data enters one bit at a time through the serial input.

After sufficient clock pulses, the data appears at the outputs:

- Q1
- Q2
- Q3
- Q4

Thus, serial data is converted into parallel data.

9.4 Applications

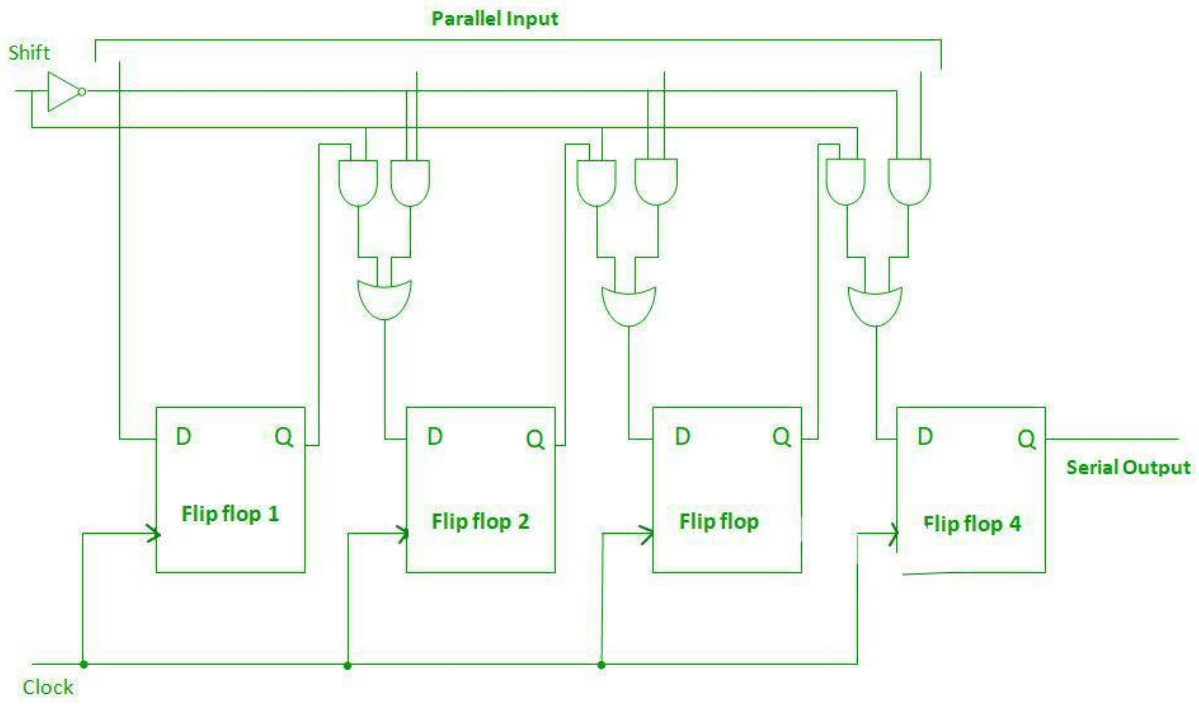
- Serial-to-parallel conversion
- Communication interfaces
- Data reception systems

10. PARALLEL-IN SERIAL-OUT (PISO) SHIFT REGISTER

10.1 Definition

A **Parallel-In Serial-Out (PISO)** shift register accepts data in **parallel form** and shifts it out **serially**.

10.2 Block Diagram



10.3 Working

- All bits are loaded into the register at the same time
- On each clock pulse, one bit is shifted out through the serial output

Thus, parallel data is converted into serial data.

10.4 Applications

- Parallel-to-serial conversion
- Data transmission systems

---

11. PARALLEL-IN PARALLEL-OUT (PIPO) SHIFT REGISTER

11.1 Definition

A **Parallel-In Parallel-Out (PIPO)** shift register loads data in parallel and provides output in parallel.

11.2 Block Diagram

P3 --> [FF1] --> Q3
P2 --> [FF2] --> Q2
P1 --> [FF3] --> Q1
P0 --> [FF4] --> Q0

11.3 Working

- All bits are entered at the same time
- All outputs are available simultaneously

This provides very fast data loading and retrieval.

11.4 Applications

- Temporary storage
- Buffer registers
- Internal processor data handling

---

12. BIDIRECTIONAL SHIFT REGISTER

12.1 Definition

A **bidirectional shift register** is a shift register in which data can be shifted in **both left and right directions**.

12.2 Working

A control input determines the direction of shifting:

- **0 → Shift Right**
- **1 → Shift Left**

12.3 Applications

- Arithmetic operations
- Data manipulation
- Processor design

---

13. APPLICATIONS OF SHIFT REGISTERS

Shift registers are used for:

- Data storage
- Data transfer
- Serial-to-parallel conversion
- Parallel-to-serial conversion
- Sequence generation
- Communication systems

---

14. INPUT / OUTPUT (I/O) ORGANIZATION

14.1 Introduction

A computer communicates with the external world using **Input/Output (I/O) devices**.

These devices help the computer:

- receive data
- process data
- provide results

14.2 Meaning of I/O

I/O = Input / Output

14.3 Examples of I/O Devices

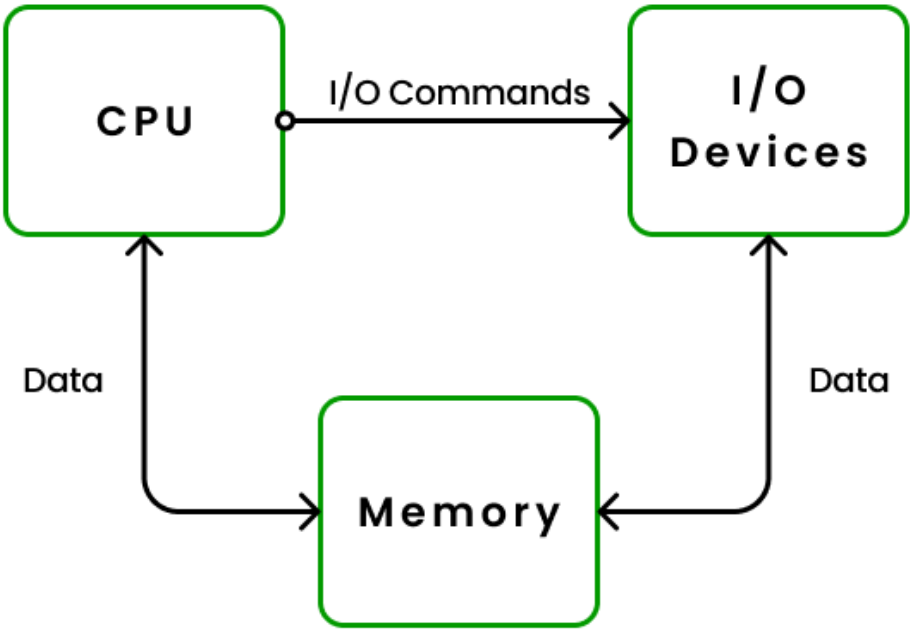
Input Devices

- Keyboard
- Mouse
- Scanner
- Microphone

Output Devices

- Monitor
- Printer
- Speaker
- Projector

14.4 Basic I/O Communication Model



14.5 Working of I/O Model

1. The input device sends data to the CPU
2. The CPU processes the data
3. The output device displays or presents the result

Example

Keyboard → CPU → Monitor

15. INTERRUPTS

15.1 Introduction

In a computer system, some events need immediate CPU attention.
For example:

- a key is pressed
- data transfer is complete

- printer requires service

Instead of continuously checking all devices, the CPU uses **interrupts**.

15.2 Definition

An **interrupt** is a signal sent to the CPU indicating that an event has occurred and requires immediate attention.

15.3 Need for Interrupts

Without interrupts, the CPU must repeatedly check devices.
This wastes time and reduces efficiency.

Interrupts help the CPU respond **only when necessary**.

15.4 Interrupt Handling Process

When an interrupt occurs, the CPU performs the following steps:

1. Completes the current instruction
2. Saves the current program state
3. Jumps to the **Interrupt Service Routine (ISR)**
4. Executes the ISR
5. Restores the previous state
6. Returns to the main program

15.5 Interrupt Flow Diagram

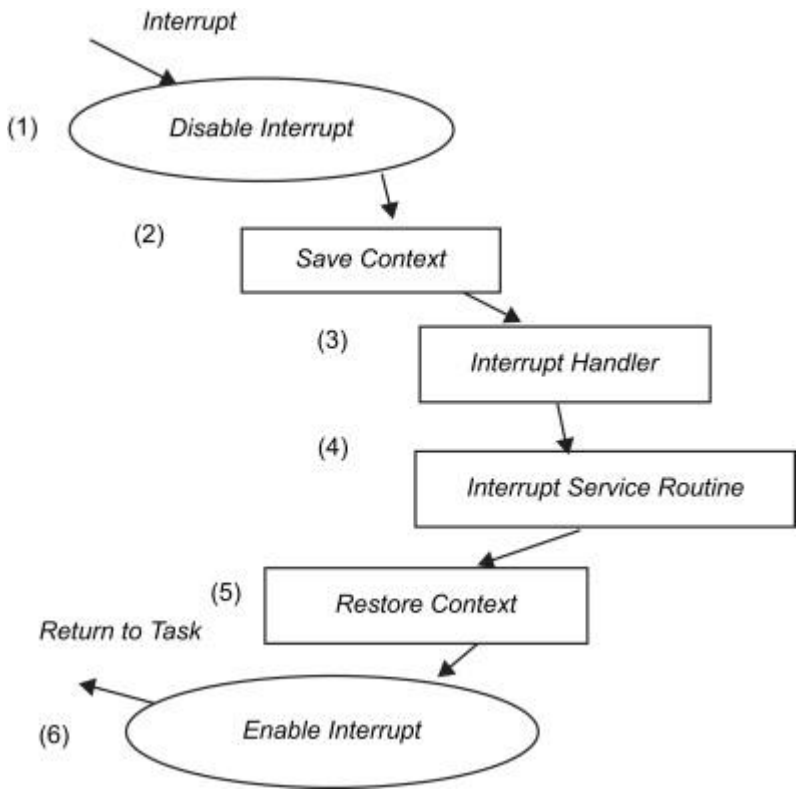
Program Execution
↓
Interrupt Occurs
↓
Save Current State
↓
Execute ISR
↓
Return to Main Program

15.6 Interrupt Hardware

Interrupt hardware refers to the circuitry used to:

- detect interrupt requests
- send interrupt signals to CPU
- identify the source of interrupt

Simple Block Diagram



15.7 Advantages of Interrupts

- Better CPU utilization
- Faster response to external events
- Reduced unnecessary checking

---

16. DIRECT MEMORY ACCESS (DMA)

16.1 Introduction

For large data transfers, it is inefficient for the CPU to handle every data item between memory and I/O devices.

To solve this, systems use **Direct Memory Access (DMA)**.

16.2 Definition

DMA is a method by which data is transferred directly between an **I/O device** and **memory** without continuous CPU involvement.

16.3 Need for DMA

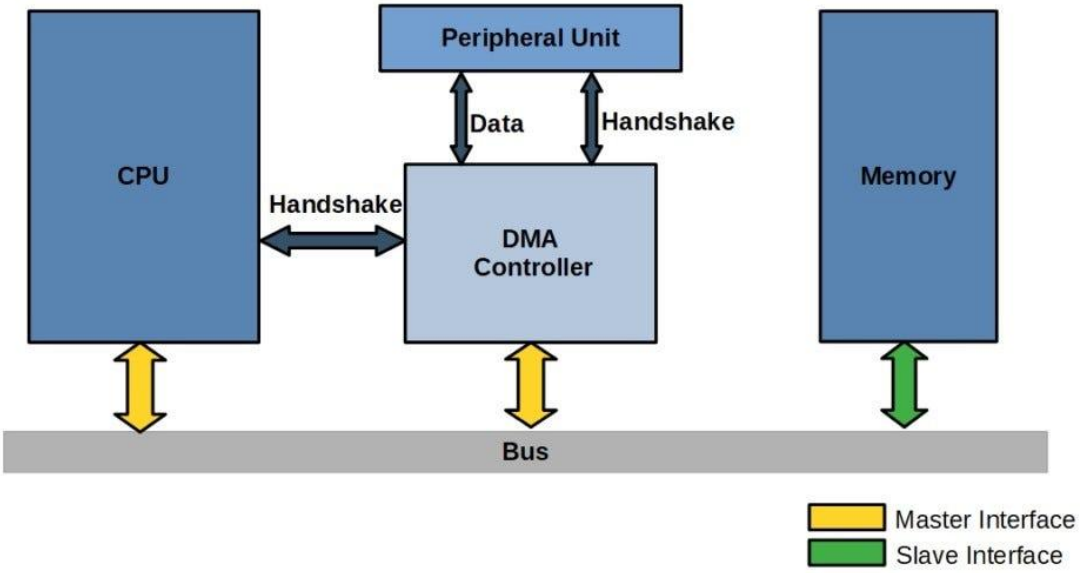
Without DMA:

- CPU must control every data transfer
- system becomes slower

With DMA:

- data transfer is faster
- CPU is free to perform other tasks

16.4 DMA Block Diagram



16.5 Working of DMA

1. CPU initializes the DMA controller
2. DMA controller requests control of the system bus
3. DMA transfers data directly between memory and I/O device
4. After transfer completion, DMA informs the CPU

16.6 Advantages of DMA

- High-speed data transfer

- Reduced CPU workload
- Efficient for large data blocks

16.7 Applications of DMA

- Hard disk operations
- Audio/video streaming
- Network data transfer
- High-speed peripheral communication

---

17. PROGRAMMED I/O AND DMA – COMPARISON

Feature	Programmed I/O	DMA
CPU involvement	High	Low
Speed	Slow	Fast
CPU workload	High	Low
Suitable for	Small data transfer	Large data transfer

Conclusion

DMA is preferred when large amounts of data must be transferred efficiently.

---

18. BUSES

18.1 Introduction

All major components of a computer system such as:

- CPU
- Memory
- I/O devices

must communicate with one another.
This communication is carried out through **buses**.

18.2 Definition

A **bus** is a communication path used to transfer data, addresses, and control signals between system components.

18.3 Types of Buses

The three main types of buses are:

1. **Data Bus**
2. **Address Bus**
3. **Control Bus**

18.4 Data Bus

The **data bus** carries the actual binary data between CPU, memory, and I/O devices.

Feature

- Usually **bidirectional**

18.5 Address Bus

The **address bus** carries the address of the memory location or I/O device being accessed.

Feature

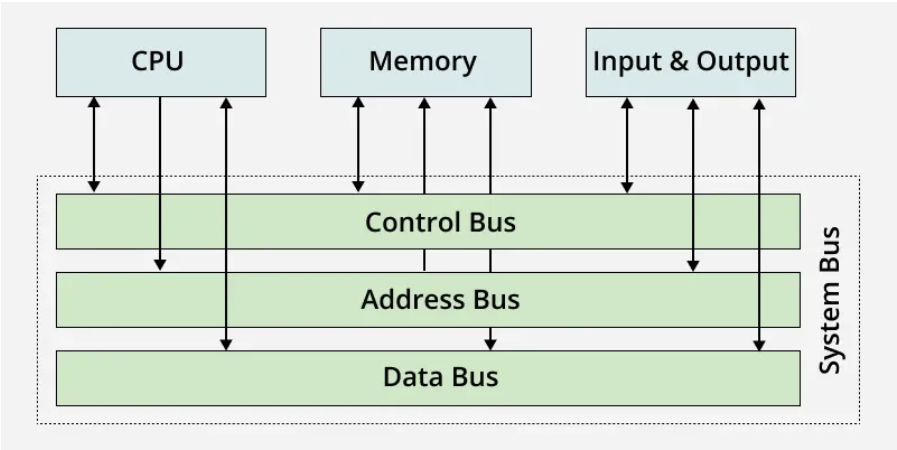
- Usually **unidirectional**
- Sent from CPU to memory or I/O

18.6 Control Bus

The **control bus** carries control and timing signals such as:

- Read
- Write
- Clock
- Interrupt
- Reset

18.7 Basic Bus Structure



19. COMMON BUS SYSTEM AND BUS ARBITRATION

19.1 Common Bus System

In many computer systems, multiple devices share a **single common bus**.

Examples of devices connected to a common bus:

- CPU
- Memory
- Keyboard interface
- Printer interface

19.2 Problem in a Common Bus System

If two or more devices attempt to use the bus at the same time, a **conflict** occurs.

This problem must be solved for proper system operation.

19.3 Bus Arbitration

Bus arbitration is the process of deciding which device gets control of the bus when multiple devices request it simultaneously.

19.4 Importance of Bus Arbitration

- Prevents bus conflicts
- Ensures orderly communication

- Improves system reliability

---

20. PIPELINING

20.1 Introduction

In a simple processor, one instruction is completed before the next instruction begins.
This limits performance.

To improve processor speed and efficiency, **pipelining** is used.

20.2 Definition

Pipelining is a technique in which multiple instructions are processed simultaneously in different stages.

It is similar to an **assembly line**, where different tasks are performed at different stages at the same time.

20.3 Need for Pipelining

Pipelining improves:

- instruction throughput
- CPU efficiency
- execution speed

20.4 Basic 3-Stage Instruction Pipeline

A simple pipeline consists of the following stages:

1. **Fetch (F)** – Fetch instruction from memory
2. **Decode (D)** – Decode or interpret the instruction
3. **Execute (E)** – Perform the required operation

20.5 Pipeline Diagram

Instruction 1 --> Fetch --> Decode --> Execute
Instruction 2 ---------> Fetch --> Decode --> Execute
Instruction 3 ----------> Fetch --> Decode --> Execute

20.6 Timing of Three Instructions

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch	-	-
2	Decode	Fetch	-
3	Execute	Decode	Fetch
4	-	Execute	Decode
5	-	-	Execute

20.7 Interpretation

At **clock cycle 3**:

- Instruction 1 is in **Execute**
- Instruction 2 is in **Decode**
- Instruction 3 is in **Fetch**

Thus, multiple instructions are processed simultaneously in different stages.

20.8 Advantages of Pipelining

- Increases instruction throughput
- Improves CPU performance
- Makes better use of processor resources

20.9 Limitation

If one stage is delayed, the subsequent instructions may also be delayed.